

Crossing the Architectural Barrier: Evaluating Representative Regions of Parallel HPC Applications

ARM

Alexandra Ferrerón (University of Zaragoza), Radhika Jagtap,
Sascha Bischoff, **Roxana Ruşitoru (ARM)**

Senior Research Engineer
Software & Large Scale Systems Research

ISPASS
April 24, 2017

Premise

Design space exploration and sensitivity studies

- Exascale, perf/W, etc.

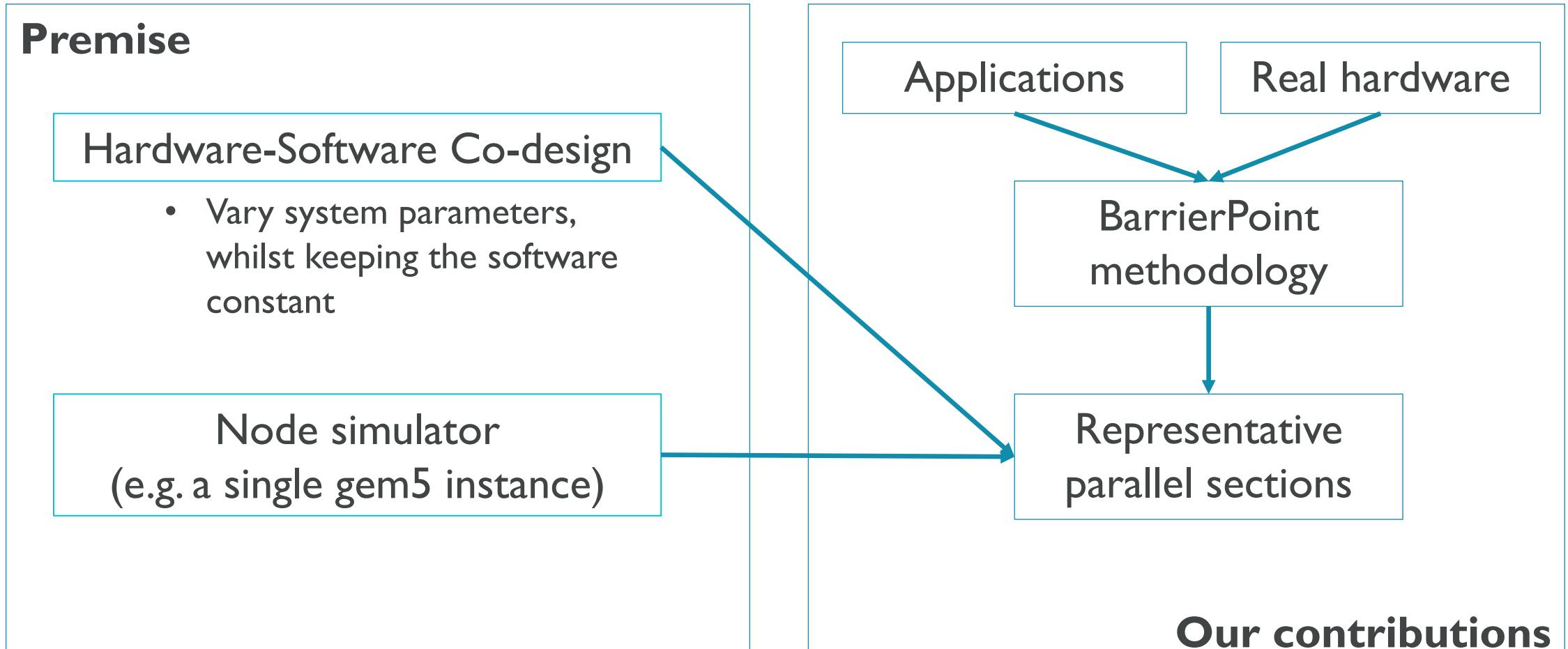
Simulation is expensive

- Accuracy versus speed



Find and run only
representative parts of parallel
applications
(ideally as little as possible!)

What it's for (example)



Currently, this methodology is intended for node-level studies.

The methodology (prior art)

- Barrier-synchronized (e.g., OpenMP) applications are an important subset of HPC parallel workloads
 - Barriers offer a natural synchronization point where all threads align!
- BarrierPoint: Sampled simulation of multi-threaded applications [Carlson et al., ISPASS'2014]
 - Simulate a select number of representative inter-barrier regions (barrier points) and predict the total application performance from those barrier points.
- We have implemented and validated the methodology for x86_64/ARMv8 using HPC benchmarks, on real hardware.

What is a barrier point (BP)?

```
/* .. A, B, and C are initialize with  
pseudo-random values .. */
```

```
BEGIN_OF_ROI();
```

```
for (itr = 0 ; itr < 3 ; itr++) {
```

```
    printf("I am part of a BP as well!");
```

```
    #pragma omp parallel for private(j,k) shared(A,B,C)
```

```
    // C <- C + A x B
```

```
    for (i = 0; i < size; ++i) {
```

```
        for (j = 0; j < size; ++j) {
```

```
            for (k = 0; k < size; ++k) {
```

```
                C[i][j] += A[i][k] * B[k][j];
```

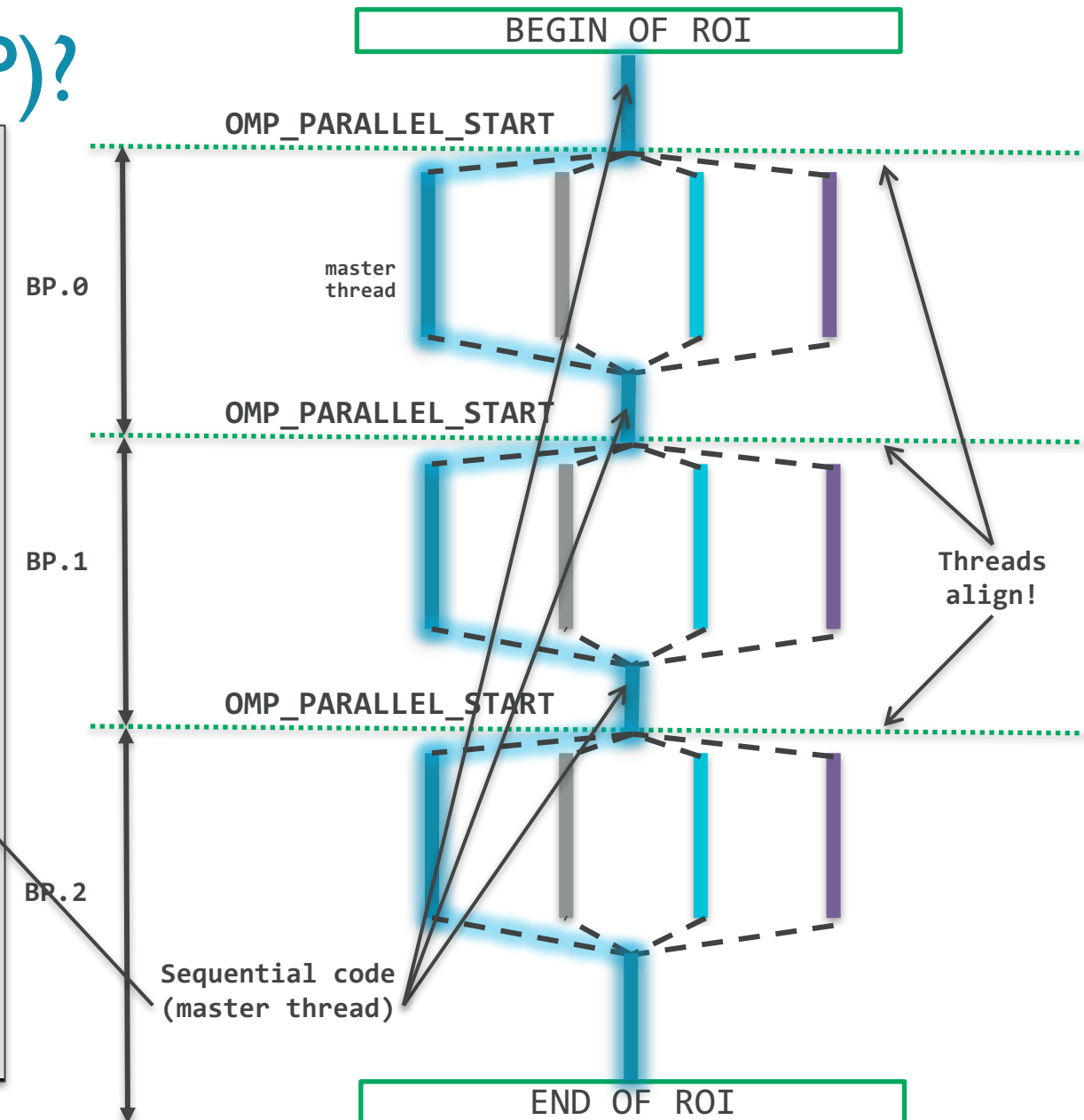
```
            }
```

```
        }
```

```
    }
```

```
}
```

```
END_OF_ROI();
```



Main contributions

- Provide an independent evaluation of the BarrierPoint methodology
 - Tested on HPC proxy applications on x86_64 and ARMv8, on real hardware
- Show that abstract characteristics can be used to identify the representative parts of HPC applications across both ISAs
- Look into architectural features (vector capabilities), and evaluate their impact on the representativeness of the selected phases
 - Errors are similar to the non-vectorised cases

Results sneak peak:

Performance estimation error <2.3% (x86_64 & ARMv8) for cycles and instructions

Total instruction count reduction of up to 178x

Cross-architectural evaluation methodology

x86_64

(1) Obtain barrier points

(2) Gather performance counter statistics

(3) Program behaviour reconstruction

(4) Barrier point set validation

AArch64

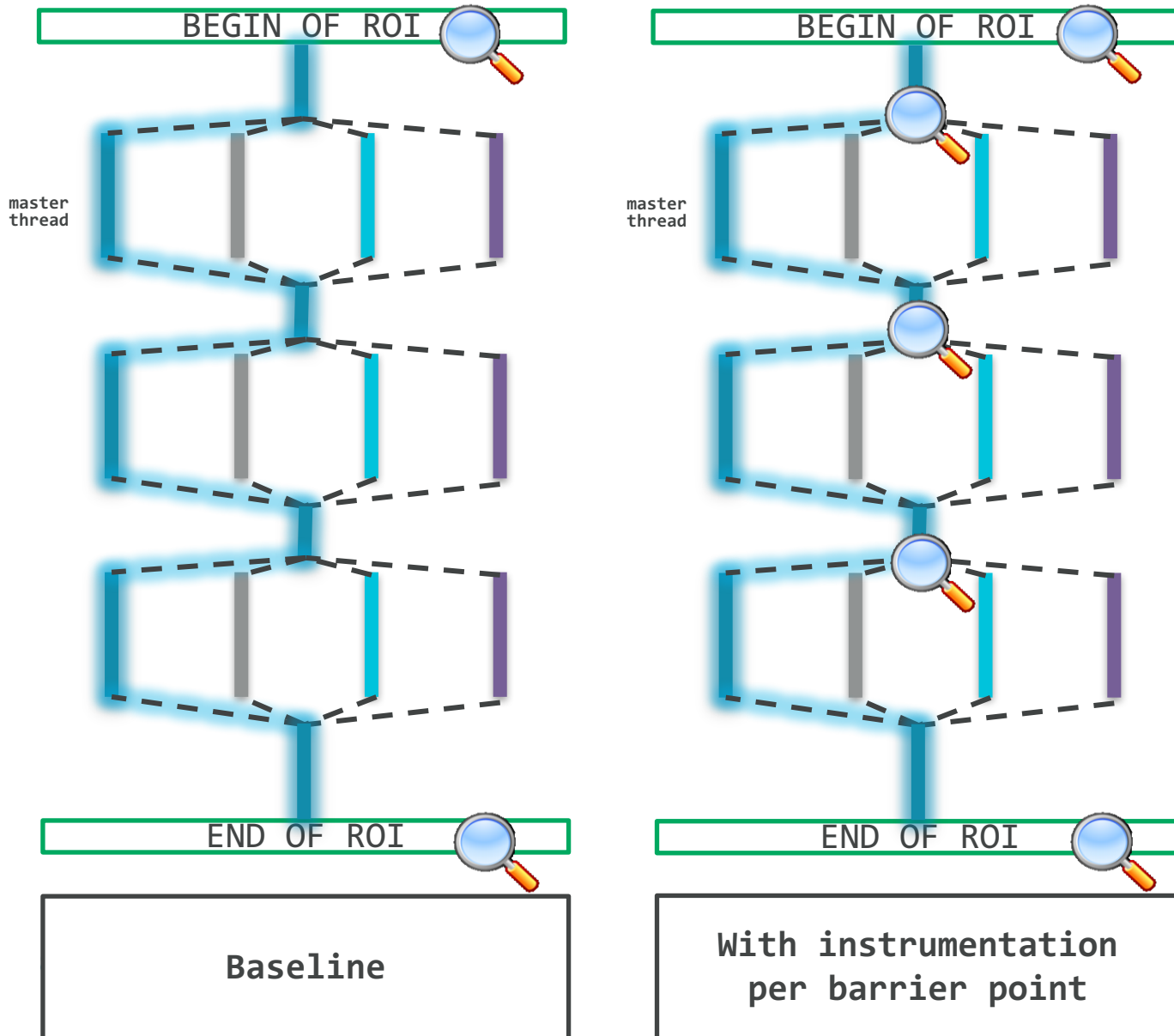
(2) Gather performance counter statistics

(3) Program behaviour reconstruction

(4) Barrier point set validation

The source code instrumentation is architecture agnostic.

Barrier point instrumentation and validation



- Adding instrumentation at the beginning of **each** parallel region
- No warm-up issues: complete application run

Experimental configuration #1

Applications

AMGMk
CoMD
Graph500
HPCG
HPGMG-FV
LULESH
MCB
miniFE
Pathfinder
RS & XSBench

Hardware

x86_64: Intel Core i7-3770 @ 3.4 GHz
(4 cores x 2 threads) 32 KB L1D+32 KB
L1I, 256 KB L2 per core 8 MB shared L3

ARM: ARMv8 AppliedMicro X-Gene @
2.4 GHz (4 clusters x 2 cores) 32 KB
L1D + 32 KB L1I per core, 256 KB L2
per cluster 8 MB shared L3 (**64-bit**)

Experimental configuration #2

Configurations:

- 1, 2, 4, 8 threads
- Vectorised/Non-vectorised
- x86_64/ARMv8

Metrics:

- Cycles, Instructions, L1-D misses, L2-D misses
- Representativeness: estimation error for performance counter metrics within acceptable bounds (5-10%).

Obtain barrier points
(10 sets per configuration, x86_64 only)

Gather performance counter
statistics
(20x+warm-up per configuration)

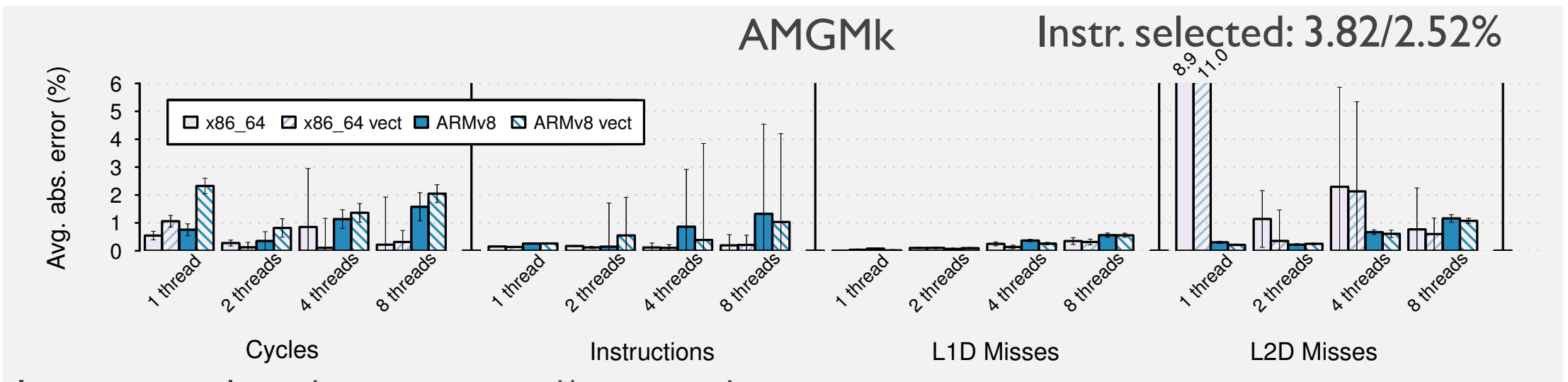
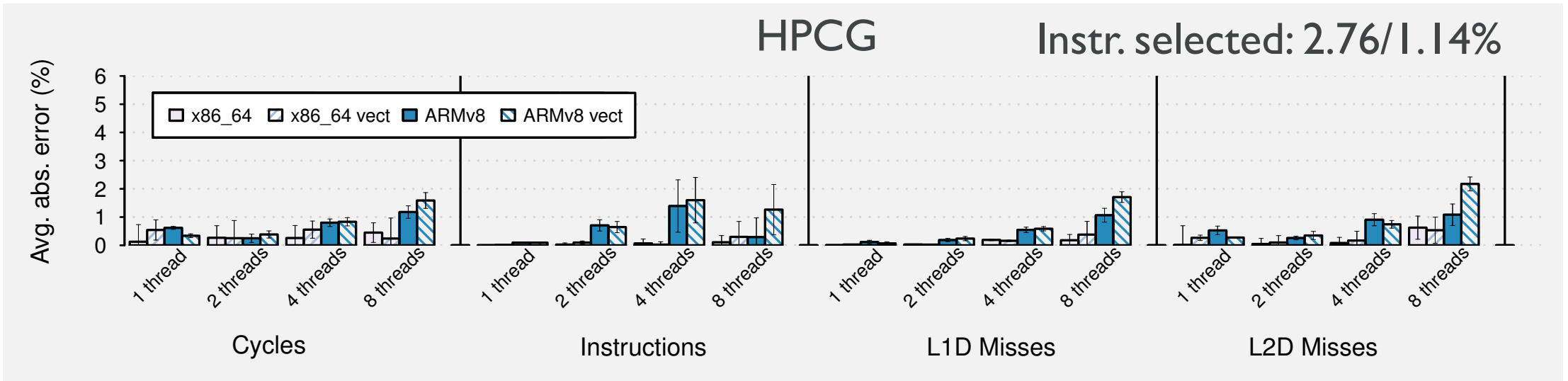
Program behaviour reconstruction
(10x per configuration)

Barrier point set validation
(10x per configuration)

Limitations

- Applications with a single large parallel region
 - There is no speedup to be gained
 - Apps: XSBench, RSBench and Pathfinder
- Statistics collection overhead
 - An issue when the size of the barrier point is too small
 - Apps: HPGMG-FV and LULESH
- Statistics collection variability
 - Number of <stat> is too low, thus the variability impact is higher
 - Apps: CoMD LID\$ misses on AArch64
- Instruction count variability due to floating-point operation accuracy
 - Apps: HPGMG-FV

Results (when it works)

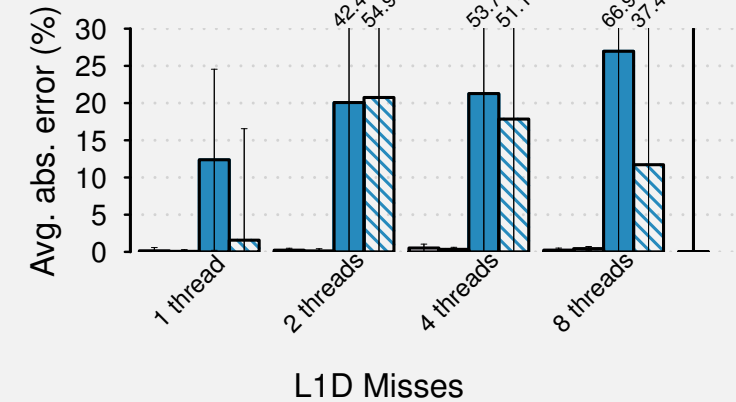
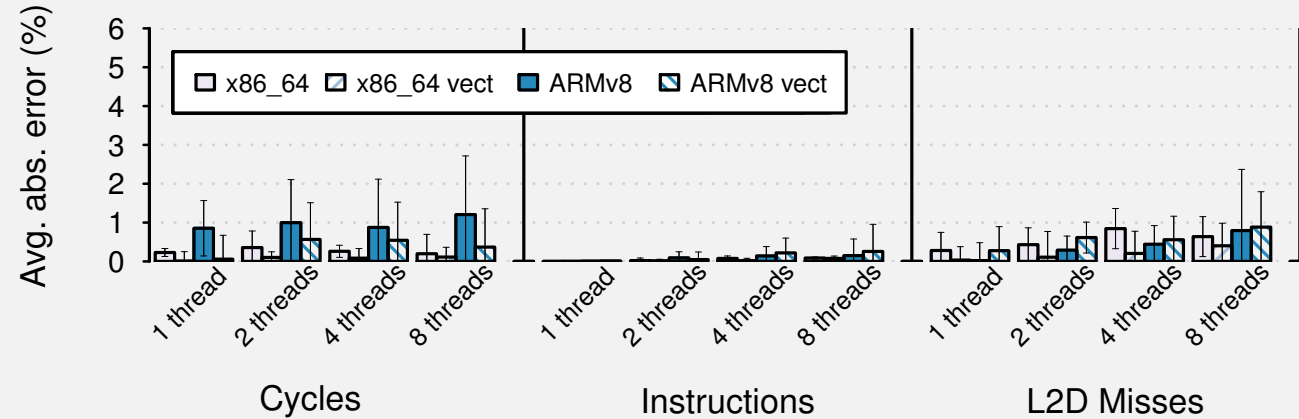


Instruction selected: non-vectorised/vectorised

Results #2 (when it works)

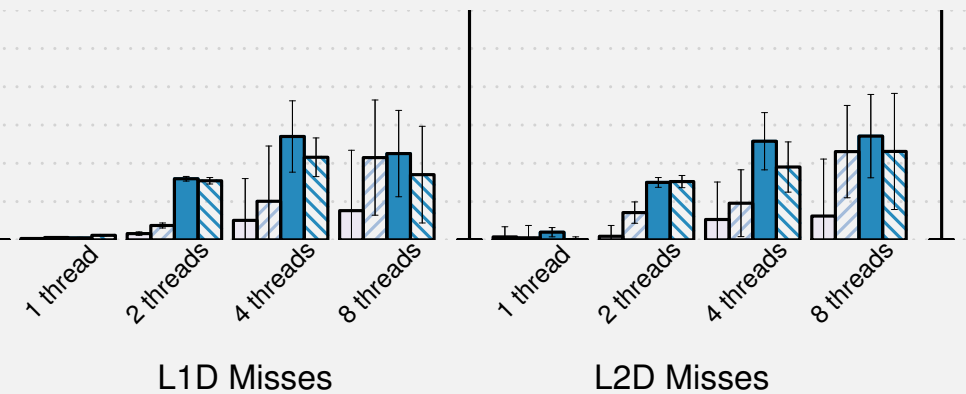
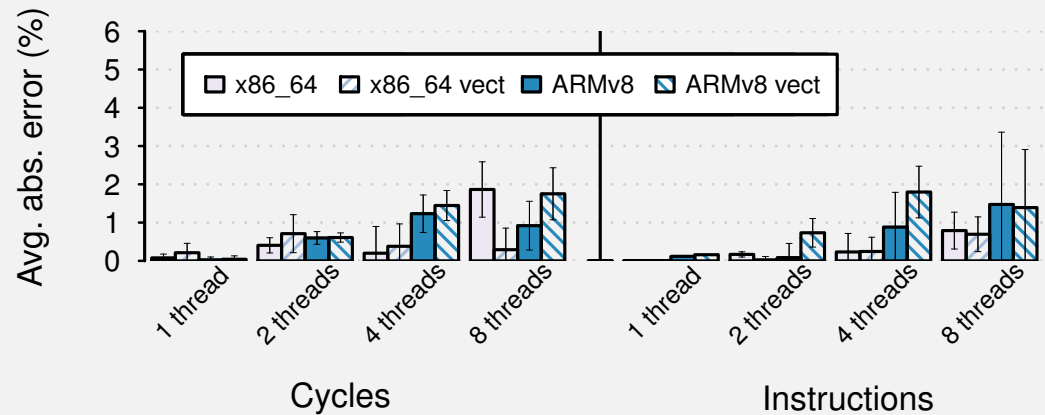
CoMD

Instr. selected: 2.07/1.42%



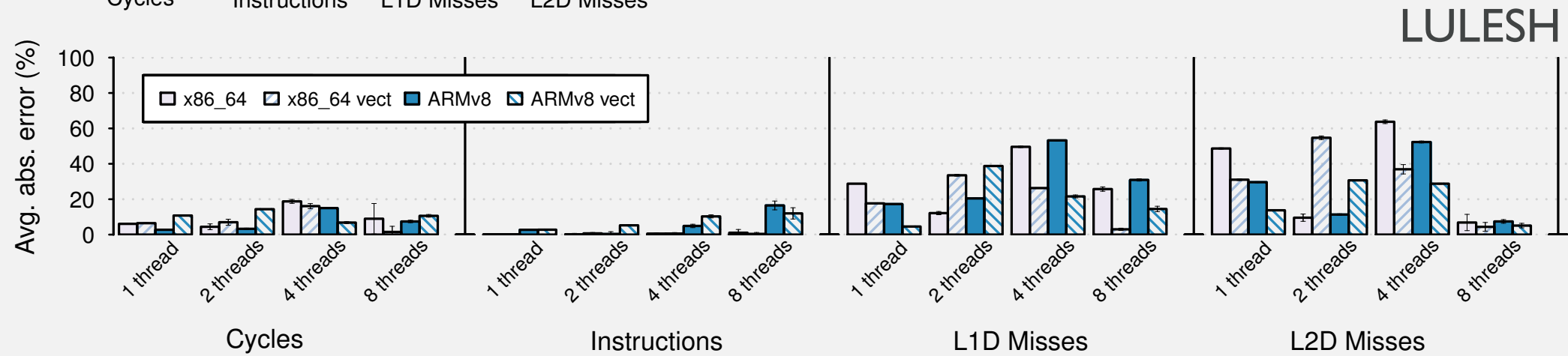
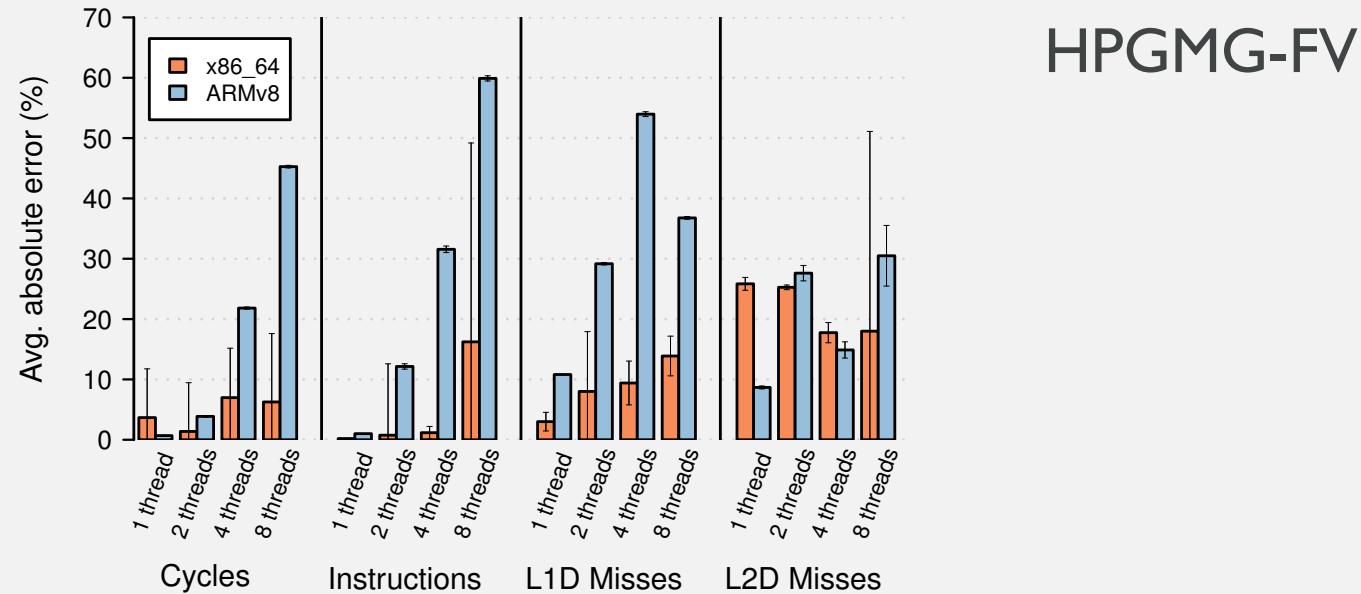
miniFE

Instr. selected: 0.56/0.59%



Instruction selected: non-vectorised/vectorised

Results (when it doesn't work)



Conclusions

- Independent evaluation of the BarrierPoint methodology
 - HPC proxy applications
 - x86_64 and ARMv8
 - AVX and NEON
- Results show that we can identify representative regions on x86_64 and validate them on ARMv8, with an error of within 3% for all statistics (exception: CoMD LID misses on ARM).
- Instruction count reduction from 2x to 178x.

Future work

- Evaluate the applicability of the methodology across different core types, such as in-order versus out-of-order.
- Validate the representative sections against a more comprehensive set of performance counters.
- Adjust the size of barrier points so that more applications benefit from the BarrierPoint methodology, such as RSBench, XSBench, and LULESH.
- Generalise the implementation to work on non-OpenMP applications.
- Quantify cross-architectural ISA differences, and explore the methodology's cross-architectural applicability limits.

Ferrerón was supported in part by grants gaZ: T48 research group (Aragón Gov. and European ESF), TIN2013-46957- C2-I-P, TIN2016-76635-C2-I-R, Consolider NoE TIN2014- 52608-REDC (Spanish Gov.) and HiPEAC-3 NoE (European FET FP7/ICT 287759). Ruşitoru has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement N° 671697.

Thank you!

Questions?